

软件体系结构实验指导书

v4.0 (2025 修订版) · 附实验报告模板

适用专业：软件工程

学期：第 6 学期

实验学时：16 学时

目录

修订说明

实验项目 1：架构质量实践之可读性与可修改性

1. 实验目标
2. 实验内容
3. 支撑的课程目标
4. 实验要求
5. 编程环境
6. 实验报告要点
7. 实验示例大纲

实验项目 2：架构质量实践之可调试性与可测试性

实验项目 3：架构质量实践之性能

实验项目 4：架构质量实践之可扩展性

实验报告模板

模板使用说明

AI 辅助写作规范

修订说明 (v4.0)

本版指导书 (v4.0) 在 v3.0 基础上进行了全面更新，主要修订内容如下：

v4.0 主要更新

- **AI 工具链全面更新：**将实验背景中的 LLM 工具从 GPT/DeepSeek 扩展为更通用的表述，并新增 GitHub Copilot、Cursor、Codeium 等 AI 编程助手的引导，同时引入 Agentic AI（智能体 AI）概念。
- **工程实践工具扩展：**在编程环境中新增容器化（Docker）、现代 CI/CD（GitHub Actions）、代码质量平台（GitHub Advanced Security、CodeQL）等工具选项。
- **架构设计方法补充：**在实验 1、2 中融入 ICONIX 过程思想——强调从需求到结构的推导过程，帮助学生理解“质量属性不是孤立的优化点，而是架构演化的结果”。
- **实验报告规范升级：**新增“AI 辅助写作规范”章节，明确 AI 辅助的边界、引用标注要求和学术诚信准则。
- **报告提交要求细化：**要求提交 Git 提交记录截图，以体现实验过程的增量演进。

v4.0 的核心理念是：在 AI 时代，工具在快速迭代，但工程思维（过程意识、度量意识、可追溯意识）依然是不可替代的底层能力。

实验项目 1：架构质量实践之可读性与可修改性

实验学时：4 学时

1. 实验目标

1. 理解大语言模型（LLM）及 AI 编程助手（如 GitHub Copilot、Cursor）辅助编码背景下，代码可读性与可修改性的重要意义。区分"AI 生成可用代码"与"AI 生成可维护代码"的本质差异。
2. 掌握基于软件质量属性（可读性、可修改性）对代码进行重构与优化的方法，并结合 AI 生成代码的特点进行批判式分析与改进。建立"不盲从 AI 输出"的工程判断力。
3. 熟练运用相关工具（静态分析、度量工具、CI/CD）对代码质量进行量化评价，理解如何基于指标不断迭代优化。掌握"度量—分析—改进—再度量"的持续优化闭环。

2. 实验内容

背景

你可选择使用既往完成的有一定规模的代码或项目（如之前学期的课程设计），或者基于一个由 AI（大语言模型或 AI 编程助手）辅助生成的有一定规模的代码或项目（可自行选择语言，如 C++/Java/Python/Go）。该项目包含一定业务逻辑，但可读性与可修改性方面有待评估和优化。

关于 AI 代码来源的建议（2025 更新）

除直接向 ChatGPT、DeepSeek、Claude 等对话式模型索取代码外，你也可以使用以下方式获取 AI 辅助代码：

- **GitHub Copilot**：在 IDE 中通过自动补全/tab 接受方式获得代码片段，观察其命名风格与抽象质量。
- **Cursor / Windsurf**：使用 AI 编辑器直接生成功能模块，体验"AI 原生开发"模式下的代码组织方式。

- **Agentic AI 工具**（如 Devin、OpenHands、GitHub Copilot Workspace）：体验 AI Agent 端到端生成项目的过程，重点关注其架构决策质量。

实验要求对 AI 代码进行**批判式审查**——AI 生成的代码往往"能运行"但"难维护"，这正是本实验的切入点。

a. 代码审查与初步评估

- 阅读代码，对其代码质量做初步评估：命名风格、注释完整度、模块化程度、边界处理、异常安全等。
- 记录常见问题（如过度链式调用、无注释、变量命名模糊、耦合度过高、AI 特有的"幻觉式代码"——看起来合理但存在潜在 bug 的代码）。
- 使用静态分析工具（如 SonarQube、Clang-Tidy、PMD、ESLint、**GitHub CodeQL**）量化可读性/可维护性指标，生成初步报告。

b. 重构与优化

- 结合可读性与可修改性需求，对代码中的命名、注释、结构等方面进行有必要的重构，并保证逻辑完整性。
- 修正边界条件处理，引入必要的异常处理或断言，提高安全性与可修改性。
- 将关键代码片段进行合理拆分或抽象（函数或模块），减少耦合度、增强可维护性。
- 引入 ICONIX 过程思想辅助重构：从用例行为出发，检查每个类/方法是否可追溯回需求描述。

c. 工具度量与结果对比

- 使用同样或更深入的度量工具，对重构后代码再次评估，比较数值变化（例如：圈复杂度降低、函数行数减少、可维护性指数上升、代码重复率下降等）。
- 形成前后对比报告，直观展示如何通过优化来弥补之前代码的不足。
- 建议将度量过程接入 CI/CD（如 GitHub Actions），体验"代码质量门禁"的工程实践。

d. 代码提交与评审

- 提交优化前后的版本（Git 提交），并记录主要修改点与原因。要求提供 Git 提交历史截图。
- 可选：让 AI（或班级同学）对你的重构方案进行"二次审查"，看是否仍有优化空间或遗漏。体验人机协同的代码评审流程。

e. 实验总结

- 总结关于可读性、可修改性与人机协同的思考。
- 思考职业规范、团队协作与创新能力在新一代软件开发中的定位，如何做到"AI 不可替代"。
- 给出对"AI 生成代码 + 人工审查与重构"相结合模式的个人见解。特别讨论：当 AI 可以自动重构时，人类工程师的核心价值在哪里？

3. 支撑的课程目标

- a. 掌握并实践提升架构质量（可读性、可修改性）的技术要点：结合 AI 辅助编程场景，学会从质量属性角度分析并改进软件代码。
- b. 基于架构质量属性优化软件实现方案：理解 AI 生成代码与人工重构协同，对软件质量进行量化评价，形成可行的持续改进过程。
- c. 培养分析复杂工程问题的能力：在 AI 时代的开发环境下，探究如何保证软件的可读与可改，既提高软件开发效率，又兼顾高质量架构。

4. 实验要求

- a. **熟悉 AI 生成代码特点**：了解大语言模型及 AI 编程助手的常见漏洞（命名混乱、忽略异常、安全漏洞、"幻觉式代码"——看起来正确但实际有逻辑缺陷的代码）。
- b. **理解并应用重构原则**：包括简短函数、单一职责、可维护模块、适当抽象等原则。参考 Martin Fowler《重构》中的核心手法。
- c. **掌握度量工具/分析工具**：如 SonarQube、Clang-Tidy、CPD（复制检测）、ESLint、GitHub CodeQL、Snyk（安全扫描）等。
- d. **保证报告完整、准确、规范**：包含需求描述、代码结构、重构过程、度量结果等内容。必须附 Git 提交记录截图。
- e. **实现语言不限**，但需保证在所选择的编程环境中可运行。
- f. **注重非功能性需求**：尤其可读性、可修改性（可结合其他质量属性，如安全性），需在报告中结合代码加以阐述，并通过工具给出度量结果。

5. 编程环境

表 1 实验 1 推荐编程环境

类别	推荐选项	说明
操作系统	Ubuntu / Windows / macOS / WSL	均可，推荐使用 Linux 环境便于工具链安装
编程语言	Python / C++ / Java / Go / Rust / TypeScript	建议与课程项目实际语言保持一致
IDE	VS Code + Copilot / Cursor / PyCharm / CLion	推荐使用集成 AI 助手的 IDE 体验真实开发场景
版本管理	Git + GitHub / GitLab / Gitee	必须提交历史记录截图
静态分析	SonarQube / Clang-Tidy / PMD / ESLint	至少选择一种工具生成量化报告
安全扫描	GitHub CodeQL / Snyk / Bandit (Python)	可选，用于检查 AI 代码的安全隐患
CI/CD (可选)	GitHub Actions / Jenkins	将代码质量检查自动化

6. 实验报告要点

a. 相关知识

- 对可读性、可修改性在 AI 时代的定义、重要性、常见问题。
- 选用何种度量工具，对代码质量进行量化分析的理由。
- ICONIX 过程简介：从用例模型到领域模型到类图的推导过程（简要了解，不要求完整实践）。

b. 实验对象概述

- 简要介绍 AI 生成代码的规模、主要功能、生成方式（对话式/IDE 补全式/Agent 式），展示初步检测到的问题点或风险点。
- 描述非功能性需求（可读性、可修改性）在此项目中的优先级及期望水平。

c. 系统结构 & 重构演化

- 图示重构前后代码/模块结构差异，说明每次演化的原因与效果。
- 对比 AI 原始代码与修改后代码的主要差异点（命名、抽象、异常处理、可追溯性等）。

d. 质量属性度量与结果分析

- 使用工具对重构前后代码进行度量、生成统计结果（如 McCabe 圈复杂度、重复率、可维护性指数、代码异味数等）。
- 分析数值变化，证明改进效果。
- 如使用 CI/CD 工具，展示自动化检查报告截图。

e. 代码示例

- 给出关键代码片段及注解说明，引导如何实现更好的可读性与可修改性。

f. 结论

- 对"AI + 人工重构"的反思：评价效果如何？是否大幅提升可读性？
- 讨论"当 AI 可以自动重构时，人类工程师的核心价值在哪里？"
- 若有不足或改进方案，请在结论中说明。

7. 实验示例大纲

表 2 实验 1 建议执行流程

阶段	任务	产出	建议时长
1. 初始代码获取	从 AI 工具获取或选择历史项目代码（1000 行左右为宜）	初始代码仓库	0.5 学时
2. 检测 & 记录问题	使用 SonarQube / CodeQL 扫描，记录可维护性指数、重复代码比例、代码异味等	初始度量报告	1 学时
3. 重构 & 注释	改善命名、异常处理，拆分函数，模块化抽象，Git 提交每一步	重构后代码 + 提交历史	1.5 学时
4. 复测 & 对比	再次使用度量工具验证指标变化，定性分析可读性提升	对比报告	0.5 学时
5. 总结报告	撰写实验报告，重点阐述 AI 生成代码与人工修正的关系	实验报告	0.5 学时

实验项目 2：架构质量实践之可调试性与可测试性

实验学时：4 学时

1. 实验目标

1. 理解软件架构质量中可调试性与可测试性的关键作用，尤其在 AI 时代大量自动生成或辅助生成代码的背景下。认识到"AI 生成代码能运行"不等于"AI 生成代码能被测试"。
2. 掌握可调试性与可测试性的主要原则、方法与工具，通过实践优化已有系统的调试和测试环境。理解"可测试的架构"是一种主动设计而非事后补救。
3. 熟悉常用度量和评估技术，基于自动化测试框架和调试工具，对系统的可测试性进行量化评价，并结合可调试性策略（如日志、断言、监控、分布式追踪）来改进系统结构。

2. 实验内容

背景

你将接手一段已有一定规模的代码或项目（可以是第一实验项目中同一代码库的后续改进）——其中部分代码由 AI 辅助生成。要求你聚焦可调试性或聚焦可测试性（自选），在此系统上进行如下实践：

a. 可调试性评估与改进

- 分析现有代码的可调试性：日志系统是否完善、断言/异常处理是否合理、本地与远程调试手段是否完备。
- 对 AI 生成的模块特别关注：是否存在难以定位错误的"黑盒"逻辑？AI 代码的不可解释性是否增加了调试难度？
- 改进措施：添加合理日志（含业务关键信息、分布式追踪 ID）、使用断言或自定义异常类，提高调试效率；可结合调试器脚本或可视化调试工具进行验证。
- 若系统为多线程/分布式，引入 OpenTelemetry / Jaeger 等分布式追踪工具（可选）。

b. 可测试性评估与单元测试/集成测试编写

- 检查系统分层与模块耦合度，识别可测试性难点（如硬依赖外部服务、缺乏接口抽象、AI 代码的副作用隐藏等）。
- 根据可测试性原则，进行必要的模块解耦或接口化，并使用 Mock / Fake 组件替代外部依赖，提升测试覆盖面。
- 使用测试框架（如 GoogleTest、JUnit、PyTest、Vitest 等）编写单元测试和集成测试，对关键模块进行白盒/黑盒测试。
- 针对 AI 生成代码，特别设计"边界条件测试"和"异常路径测试"——AI 往往在"happy path"表现好，但在边界和异常处理上存在漏洞。

c. 自动化测试度量与结果对比

- 通过覆盖率工具（如 gcov、Jacoco、Coverage.py、llvm-cov）度量测试覆盖率（行覆盖、分支覆盖等）。
- 对比改进前后测试覆盖率以及调试时间或定位问题的效率变化，生成度量报告。
- 建议将测试接入 CI/CD，体验"测试门禁"的工程实践。

d. AI 对调试与测试的影响

- 结合实验中遇到的问题，讨论使用 AI 生成测试用例或辅助调试脚本的可行性。
- 评估 AI 生成的测试代码质量（如漏测、误测、边界覆盖不足），并总结人机协同改进思路。
- 讨论：当 AI 可以自动生成测试时，测试工程师的角色如何转变？

e. 实验总结

- 总结在可调试性与可测试性提升中的经验，结合团队合作、职业规范（如对日志隐私信息保护）等进行反思。
- 说明在 AI 辅助下仍需保持主动思考和团队协作，以保障软件质量。

3. 支撑的课程目标

- a. 掌握可调试性与可测试性的知识与实作要点：在 AI 时代的开发环境下，能够分析并改进系统调试和测试结构。

- b. 具备面向架构质量属性的优化和评价能力：可使用自动化工具对系统可测试性进行量化度量，并根据结果采取针对性重构措施。
- c. 提升分析复杂软件工程问题的能力：通过问题导向、迭代式学习，对实验项目中的调试难点和测试盲区进行探究式解决。

4. 实验要求

- a. 理论与实践相结合：在实验前，阅读可调试性、可测试性相关资料，了解其中的关键概念与常用技巧（断言、日志、Mock 测试、契约式编程等）。
- b. 解决实际问题：从已有系统中找出调试和测试上的不足（如缺少日志信息、模块紧耦合、测试覆盖率低），并着手改进。
- c. 度量指标与工具应用：选择合适的度量工具（如覆盖率工具、SonarQube 等），评估改进效果。
- d. 报告规范：要求报告中含需求描述、系统结构图、非功能性需求阐述、改进策略、度量结果等。
- e. 实现语言不限：Python / C++ / Java / Go / Rust / TypeScript 等，但要保证在相应开发环境下可正确运行。
- f. 深度讨论可调试性或可测试性：至少选取其中一方面做深入探究，引用代码示例。
- g. 与代码密切结合：需展示改进前后的关键代码，对新增日志、测试、断言等进行示例说明。
- h. 使用工具对架构质量属性进行量化度量：将度量结果与改进方案对应解释。

5. 编程环境

表 3 实验 2 推荐编程环境

类别	推荐选项	说明
操作系统	Ubuntu / Windows / macOS / WSL	均可
编程语言	Python / C++ / Java / Go / Rust / TypeScript	与实验 1 保持一致最佳
IDE	VS Code / Cursor / PyCharm / CLion	推荐使用支持 AI 调试功能的 IDE
覆盖率工具	gcov / Jacoco / Coverage.py / llvm-cov / Istanbul	按语言选择
日志框架	Log4j / spdlog / Python logging / winston	按语言选择
Mock 测试	GoogleTest + gMock / Mockito / PyTest-mock / Vitest	按语言选择
追踪（可选）	OpenTelemetry / Jaeger / Zipkin	分布式系统可选
CI/CD（可选）	GitHub Actions	将测试自动化

6. 实验报告要点

a. 相关知识

- 可调试性的含义（日志、断点调试、异常捕获、监控、分布式追踪等）。
- 可测试性的含义（单元测试、集成测试、Mock 测试、覆盖率、契约测试等）。
- 可测试架构设计原则：依赖注入、接口隔离、控制反转（IoC）。

b. 需求描述

- 重点描述非功能性需求——可调试性、可测试性的具体期望（如"出现错误时能在 5 秒内定位问题"或"测试覆盖率需达 80%"等）。

c. 系统/模块结构

- 如有结构变动，可分步展示演化过程，说明为何这样调整能增强可测试性/可调试性。

- 若引入了依赖注入、接口抽象等架构改动，需图示说明。

d. 质量属性实现及评价

- 列举改进点：新增日志、断言、抽象层解耦等，并结合实际代码说明。
- 展示改进前后的度量结果（如日志覆盖点、测试用例数量、覆盖率提升等）。

e. 关键代码片段

- 给出重构或新增的核心测试用例、日志/断言插入示例，以及对应解释。

f. 实验结论

- 评价 AI 辅助情况下，人类工程师在调试和测试体系建设中的主导作用。
- 结合团队协作、责任意识、职业规范，阐述本次实践对可调试性与可测试性的认识。

实验项目 3：架构质量实践之性能

实验学时：4 学时

1. 实验目标

- 理解软件架构质量中性能的核心作用，掌握影响软件性能的关键因素，包括计算复杂度、I/O 速度、并发控制、内存布局、缓存友好性等。
- 掌握现代高性能架构设计方法，如缓存策略、并行计算、非阻塞编程、向量化计算等，并学习如何避免常见的性能瓶颈。
- 使用性能分析工具进行优化，结合实际系统，使用 Profiling（性能分析）、Benchmarking（基准测试）等工具进行量化评估与改进。建立"先度量再优化"的科学方法论，避免盲目优化。

2. 实验内容

背景

你将接手一个已有一定规模的系统（可以是前几个实验中的代码项目），并针对其性能瓶颈进行深入优化。本次实验强调**数据驱动优化**——所有优化决策必须有 Profiling 数据支撑，禁止凭直觉优化。

a. 性能问题分析

- 通过 Profiling 工具（如 perf、gprof、VTune、Flamegraph、**Scalene** (Python)、**pprof** (Go)）分析 CPU 占用、内存消耗、I/O 负载、锁竞争等关键性能指标。
- 确定性能瓶颈，并分析导致性能下降的主要原因（如循环嵌套、数据库查询慢、锁竞争、内存分配频繁、缓存未命中等）。
- 使用 Flamegraph 或类似的火焰图工具可视化性能热点，直观识别耗时函数。

b. 性能优化策略设计

- **数据结构优化**：选择更合适的数据结构（如哈希表替换线性表、使用缓存、预分配内存、避免内存碎片等）。
- **计算优化**：减少不必要的计算，避免 $O(n^2)$ 及以上的复杂度算法；考虑向量化计算（SIMD）或 GPU 加速（可选）。
- **I/O 优化**：批量处理数据，减少磁盘/网络 I/O 频率；使用零拷贝技术；引入异步 I/O。
- **并发优化**：使用多线程、多进程、协程（coroutine）、线程池、无锁数据结构等方式，提高系统吞吐量。
- **缓存优化**：引入 LRU、LFU 等缓存策略；利用 CPU 缓存局部性原理优化数据访问模式。

c. 优化实现与验证

- 选择合适的优化方案并进行代码改进，确保优化方案可行。
- 通过 Benchmarking 工具（如 Google Benchmark、JMH、timeit、criterion.rs）量化优化效果，并记录改进前后的性能数据。
- 对比不同优化策略的效果，分析每种策略的收益与代价（如代码复杂度增加、可维护性降低等）。

d. AI 对性能优化的影响

- 讨论 AI 生成的代码在算法复杂度、资源使用、缓存友好性等方面是否存在低效情况，并提供优化方案。
- 评估 AI 代码优化能力的局限性：AI 往往生成"正确但未必高效"的代码，对性能敏感场景缺乏深度优化。
- 讨论人类开发者如何结合 AI 进行高效性能优化——AI 可以快速生成 benchmark 框架，人类负责解读数据并制定策略。

e. 实验总结

- 反思性能优化的挑战：高性能架构需要长期积累和深入理解，不能只依赖 AI 自动优化。
- 高性能计算的社会价值：如 AI 训练、高速金融计算、科学计算等领域对高性能软件的需求。
- 总结"过早优化是万恶之源"与"不度量就优化是盲目"之间的平衡。

3. 支撑的课程目标

- 掌握高性能架构优化策略，提高软件运行效率。
- 能够使用 Profiling & Benchmarking 工具，进行性能度量和优化。
- 培养工程思维与职业精神，理解高性能计算的社会价值。

4. 实验要求

- a. 选择一个现有系统，并通过 Profiling 确定性能瓶颈。必须提供 Profiling 截图或火焰图。
- b. 至少使用 2 种不同的优化技术（如缓存、多线程、数据结构优化、异步 I/O 等）。
- c. 代码优化需包含对 AI 代码的分析与改进。
- d. 实验报告需提供优化前后的 Benchmarking 结果，并结合工具（如 Flamegraph）进行可视化分析。
- e. 分析每种优化策略的收益与代价，包括代码复杂度、可维护性、可移植性等维度的变化。

5. 编程环境

表 4 实验 3 推荐编程环境

类别	推荐选项	说明
操作系统	Ubuntu / Windows / macOS	Linux 下 Profiling 工具最完善
编程语言	C++ / Java / Python / Rust / Go	C++/Rust 更适合底层性能优化
Profiling	perf / gprof / VTune / Scalene / pprof	按语言和场景选择
火焰图	Flamegraph / speedscope.app	可视化性能热点
Benchmarking	Google Benchmark / JMH / criterion.rs / timeit	量化优化效果
内存分析	Valgrind / massif / heaptrack	检测内存瓶颈

6. 实验报告要点

- a. 相关知识：讨论高性能计算的关键因素（CPU、内存、I/O、缓存、并发）。

- b. **性能优化前后的 Benchmarking 结果：**通过数据对比优化效果。要求提供具体数值（如"响应时间从 500ms 降至 120ms"）。
- c. **优化代码示例：**展示如何提高数据结构、算法、并发处理的效率。
- d. **实验结论：**总结高性能架构优化的挑战，探讨性能优化的社会价值与工程权衡。

实验项目 4：架构质量实践之可扩展性

实验学时：4 学时

1. 实验目标

- 理解"可扩展性（Scalability）"的核心作用，掌握扩展系统容量、提高处理能力的方法。
- 选择性研究分布式系统、微服务、负载均衡、事件驱动架构等技术之一，并实践如何优化系统以支持未来增长。
- 结合 AI 时代的需求，探讨 AI 代码在可扩展性设计上的优势与局限。理解"可扩展性是一种架构级别的设计决策"。

2. 实验内容

背景

你将优化一个已有的系统，使其更具可扩展性，包括高并发处理、分布式扩展、负载均衡等改进。实验任务可考虑包括：

a. 分析当前系统的可扩展性瓶颈

- 是否单点架构（如所有请求都集中到一个数据库、一个服务实例）？
- 是否存在硬编码逻辑，导致难以增加新功能？
- 是否能够水平扩展（Scale-out）和垂直扩展（Scale-up）？
- AI 生成的架构代码是否存在扩展性局限（如单体架构、硬编码配置、缺乏服务发现机制）？

b. 设计可扩展性优化方案

- **数据库扩展：**使用读写分离、分片（Sharding）、缓存（Redis、Memcached）等优化数据库性能。
- **服务架构优化：**将单体应用拆分为微服务（Microservices）或模块化单体（Modular Monolith），使系统更易扩展。

- **负载均衡**：使用 Nginx、HAProxy、Kubernetes Ingress 等技术，优化流量分发，提高吞吐量。
- **事件驱动架构**：使用消息队列（Kafka、RabbitMQ、NATS）进行解耦，避免阻塞。
- **容器化与编排**：使用 Docker 容器化应用，使用 Kubernetes 或 Docker Swarm 进行编排（可选加分项）。

c. 优化实现与验证

- 实际修改代码，实现至少一种可扩展性优化（如数据库分片、负载均衡、微服务拆分、消息队列引入等）。
- 通过负载测试工具（Apache JMeter、Locust、k6），评估扩展性优化效果，并与优化前对比。
- 记录系统在增加并发量时的响应时间、吞吐量和错误率变化。

d. AI 对可扩展性优化的影响

- 讨论 AI 生成的架构代码是否存在扩展性局限（如单体架构、硬编码、缺乏弹性设计）。
- 评估 AI 辅助架构设计的可能性：AI 可以生成微服务模板、Dockerfile、K8s 配置，但能否做出正确的拆分决策？
- 提供人机协作的改进方案：人类负责架构决策，AI 负责配置生成和模板代码。

e. 实验总结

- 反思架构设计的长期性，强调持续学习、适应变化的必要性。
- 结合社会实际案例，如云计算、分布式 AI 计算、大规模推荐系统等，讨论可扩展性在现代系统中的作用。

3. 支撑的课程目标

- 掌握可扩展性架构优化策略，提高系统的伸缩性。
- 能够使用负载均衡、数据库优化、消息队列等技术，提升系统扩展能力。
- 培养终身学习与架构演进思维，理解软件系统随需求变化而演进的必要性。

4. 实验要求

- a. 至少优化一个系统瓶颈，提高其可扩展性。
- b. 代码优化需包含 AI 代码的扩展性分析与改进。
- c. 报告中需包含负载测试数据，并分析优化效果。要求提供 JMeter / Locust / k6 测试报告截图。
- d. 若实现容器化部署（Docker + K8s 或 Docker Compose），可作为加分项。

5. 编程环境

表 5 实验 4 推荐编程环境

类别	推荐选项	说明
操作系统	Ubuntu / macOS / WSL	Linux 环境便于容器化实践
编程语言	Java / Go / Python / Node.js	微服务常用语言
负载均衡	Nginx / HAProxy / Traefik	反向代理与流量分发
消息队列	Kafka / RabbitMQ / NATS / Redis Stream	异步解耦
缓存	Redis / Memcached	降低数据库压力
数据库	MySQL / PostgreSQL / MongoDB	分片/读写分离实践
负载测试	JMeter / Locust / k6	量化扩展性效果
容器化（可选）	Docker / Docker Compose / Kubernetes	加分项

6. 实验报告要点

- a. **相关知识**：讨论可扩展性的核心技术（微服务、数据库分片、消息队列、负载均衡、缓存策略、容器化等）。
- b. **优化方案**：解释如何增强系统的扩展能力，附架构演化图。
- c. **实验数据分析**：通过负载测试展示扩展性优化效果，包含响应时间、吞吐量、错误率曲线。
- d. **实验结论**：总结架构优化经验，讨论云原生时代可扩展性的新挑战。

实验报告模板

模板使用说明

本模板适用于四个实验项目（可读性与可修改性、可调试性与可测试性、性能、可扩展性），只需根据各自实验主题着重讨论对应属性即可。

提交规范（2025 更新）

- **报告合并：**实验一与实验二的实验报告可合并提交；实验三与实验四亦可如此处理。
- **Git 提交记录：**每份报告必须包含 Git 提交历史截图（至少展示 3 次关键提交），以体现实验过程的增量演进。
- **AI 使用声明：**若使用 AI 辅助撰写报告或生成代码，必须在报告中明确声明 AI 的使用范围和方式（详见下文"AI 辅助写作规范"）。
- **代码仓库链接：**建议提交 GitHub / GitLab / Gitee 仓库链接，便于教师查看完整代码和提交历史。

AI 辅助写作规范

学术诚信与 AI 使用边界

在 AI 时代，使用 AI 辅助学习和写作是合理的，但必须遵守以下规范：

1. **透明声明原则：**凡使用 AI（包括 ChatGPT、Claude、Copilot、Cursor 等）辅助生成报告内容、代码或图表，必须在报告附录中明确声明使用的工具、用途和范围。
2. **人类主导原则：**AI 可以辅助整理思路、润色文字、生成模板代码，但实验设计、数据分析、结论推导、批判性思考必须由学生独立完成。
3. **事实核查责任：**AI 可能生成虚假引用、错误数据或"幻觉式代码"。使用 AI 内容后，学生有责任进行事实核查，确保内容准确可靠。
4. **代码溯源要求：**AI 生成的代码片段必须在注释中标注"AI-generated"，并在报告中说明对其进行的人工审查和修改内容。

5. **禁止完全代写：**不允许将实验报告完全交由 AI 代写后直接提交。这种行为构成学术不端。

建议做法：将 AI 视为"智能助教"而非"代笔人"——用 AI 获取灵感、检查语法、生成初稿，但最终的思考、判断和修改必须来自你自己。

封面

软件体系结构
实验报告

实验题目：
院系名称：
学生姓名：
指导教师：
专业班级：
学号：
日期：

一、相关知识

1. 理论概念说明 简要阐述本次实验涉及的架构质量属性（如可读性、可修改性、可测试性、可调试性、性能、可扩展性等），并结合课程教材或文献给出其定义或学术解释。若实验中涉及 AI 时代的代码，可在此部分探讨 AI 生成代码可能存在的典型问题，与属性之间的关系（如性能瓶颈、可扩展性不足等）。可简要引入 ICONIX 过程思想：说明本次实验中的架构改进如何可追溯回需求。2. 本次实验关注的关键技术或工具 指出本次实验中使用的静态分析、Profiling、Benchmarking、负载测试等工具或库，解释它们的作用与原理。可引用教材或在线资料，说明技术选型理由（如选择 JMeter 做负载测试的原因、选择 SonarQube 的理由）。

二、需求描述

1. 功能性需求（可选） 如果已有用例图和用例文本，请简要列举主要功能。若项目规模较大，可仅提纲描述或引用之前实验资料。2. 非功能性需求（重点） 结合本实验项目的主题（可读性/可测试性/性能/可扩展性…），使用自然语言明确描述目标。如"系统需在 500 并发场景下响应时间保持在 2 秒以内"、"代码可维护性评分在 SonarQube 上应达 B 级以上"、"测试覆盖率需达 80%"等。说明这些需求对系统整体的重要性，以及为什么需要在 AI 时代格外关注此需求。

三、系统/子系统/模块/类/模块结构

1. 结构设计与演化 给出当前系统的模块或子系统图，可以是 UML 包图、组件图、类图等。如有演化过程，分阶段展示：最初版本 → 存在的问题 → 改进后的版本，对比差异与改动原因。指出 AI 生成代码在系统结构中的位置（若有），以及是否存在架构或抽象层次上的不合理之处。
2. 模式或惯用法 如果采用了观察者模式、微服务架构、读写分离、依赖注入等模式，需要阐述应用该模式的动机和对架构质量的影响（如性能或可扩展性）。

四、架构质量属性讨论

1. 所关注的质量属性 结合实验主题（例如性能或可扩展性等），着重阐述该属性的核心指标、实现思路和常见挑战。简要回顾实验目标和非功能性需求，说明如何将这些需求映射到技术上。
2. 结合代码阐述 选取关键函数或类示例，展示如何在代码层面体现所关注的属性，比如如何在 "enqueue()/dequeue()" 里做性能优化或在 "微服务通信" 中保证可扩展性。若有 AI/LLM 生成代码段，请明确指出原代码与修改后的对比，说明改进思路。
3. 工具度量与评价 使用相应工具（如 SonarQube、Flamegraph、JMeter、Benchmark 库等）获取度量结果（如复杂度、吞吐量、响应时间、可扩展性指标）。前后对比：列举改进前后指标的变化，并分析原因（如 CPU 占用降低 XX%，响应时间缩短 XX%）。必须包含工具截图或报告导出内容。

五、代码

1. 实现语言与关键技术 说明所选编程语言以及语言特性（如 C++11 并发库、Java Streams、Python asyncio）的使用情况，列举缘由。若涉及数据库或消息队列等第三方组件，也可在此说明。
2. 关键代码清单 以简短且能体现架构改进的形式贴出核心片段，如性能优化的多线程处理、可扩展性优化的微服务拆分。代码应风格统一、便于阅读、能够正确编译/运行，并在注释中说明与质量属性相关的考虑点（如 "这段使用锁分离减少锁竞争，提高性能"）。

六、结论

1. 结构设计与质量属性收获 回顾本次实验如何在架构层面提升所关注的属性（如性能或可扩展性），以及过程中遇到的挑战（如 AI 代码段低效、模块耦合过高等）。给出在设计和演化中最显著的改进收获，如 "性能提升了 30%" 或 "负载测试下可支持用户量翻倍"。
2. 其他心得 可阐述本次实验对你后续开发实践的启示，或是对 AI 辅助编程的看法。若仍有未彻底解决的问题或下次改进方向，可在此提出。
3. AI 使用声明（必填） 说明本次实验中是否使用了 AI 辅助：
 - 若有，列出使用的工具、用途（代码生成 / 报告润色 / 思路启发等）、以及对 AI 输出进行的人工审查和修改。
 - 若未使用，声明 "本实验未使用 AI 辅助"。

附录（可选）

A. Git 提交历史截图（强烈建议包含） B. 工具生成的完整报告（SonarQube / Coverage / JMeter 等） C. 代码仓库链接 D. 参考资料列表

本指导书 v4.0 由课程组编制，基于 v3.0 全面修订。如有建议或问题，欢迎反馈。